

Securing a restful API with Spring Security

written by Hong Le Nguyen, last update: 03.2026

The related Code on Github :

<https://github.com/hong1234/JwtAuthenSecuAPI>

<https://github.com/hong1234/springboot3ShopApiV3>

1) Security Filter Chain

Spring Security Integration into Spring MVC

Terminology

Filter *all* filters implement the jakarta.servlet.Filter interface
 Servlet filters filters managed by Servlet Container
 FilterChain sequence of Servlet filters

Security filters filters managed by Spring Container
 SecurityFilterChain a sequence of some Security filters

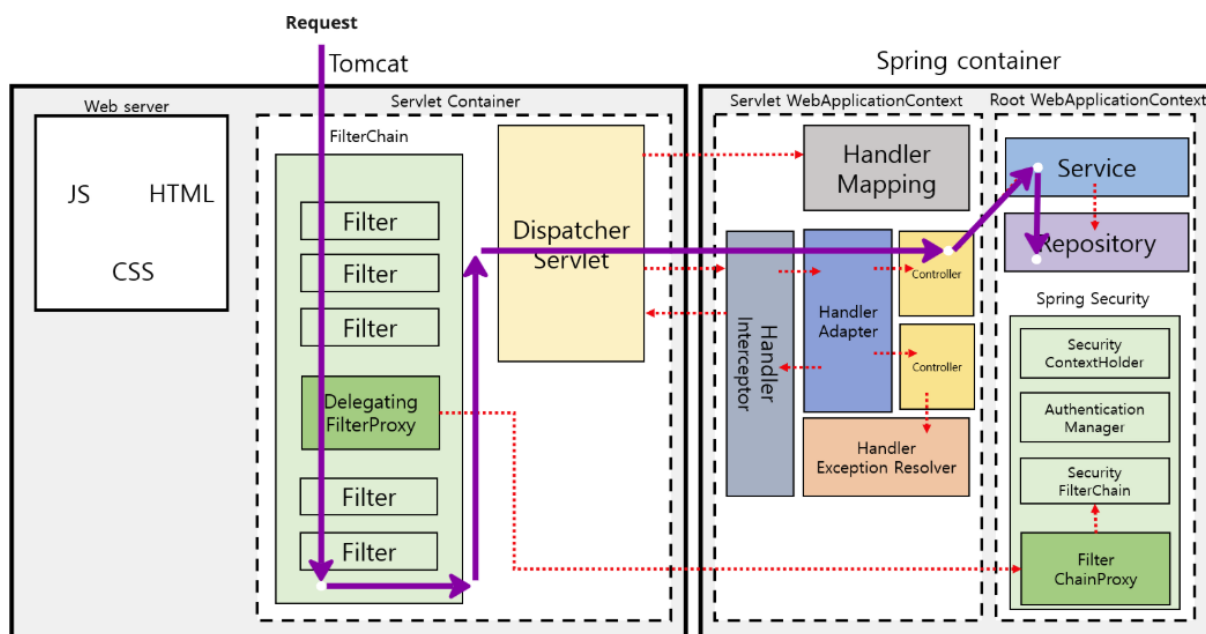


Figure 1a. Injecting Security filters into (Servlet-) FilterChain

Spring Security comes into action by adding Security filters (SecurityFilterChain) in front of the DispatcherServlet. Incoming request will visit these filters one by one before it hit DispatcherServlet and then controllers. This way, I can check Authentication, Authorization states and common attacks. Figure 1a.

DelegatingFilterProxy --

Since the Security filters are not registered via Servlet Container standards (e.g in web.xml file), the Servlet Container is not aware of them. But Security filters should be injected into the FilterChain of Servlet Container.

Spring provides a servlet filter *DelegatingFilterProxy* that acts as a bridge between the Servlet Container and the Spring Application Context. Every request will be going through this filter and *it will delegate the request to the FilterChainProxy bean named "springSecurityFilterChain"*. Figure 1a.

FilterChainProxy --

The filter bean provided by Spring Security is the entry point to the Security filters. It manages the registered SecurityFilterChains and determines which SecurityFilterChain a given request goes through based on the match between the request-path and the URL-pattern configured in SecurityFilterChains.

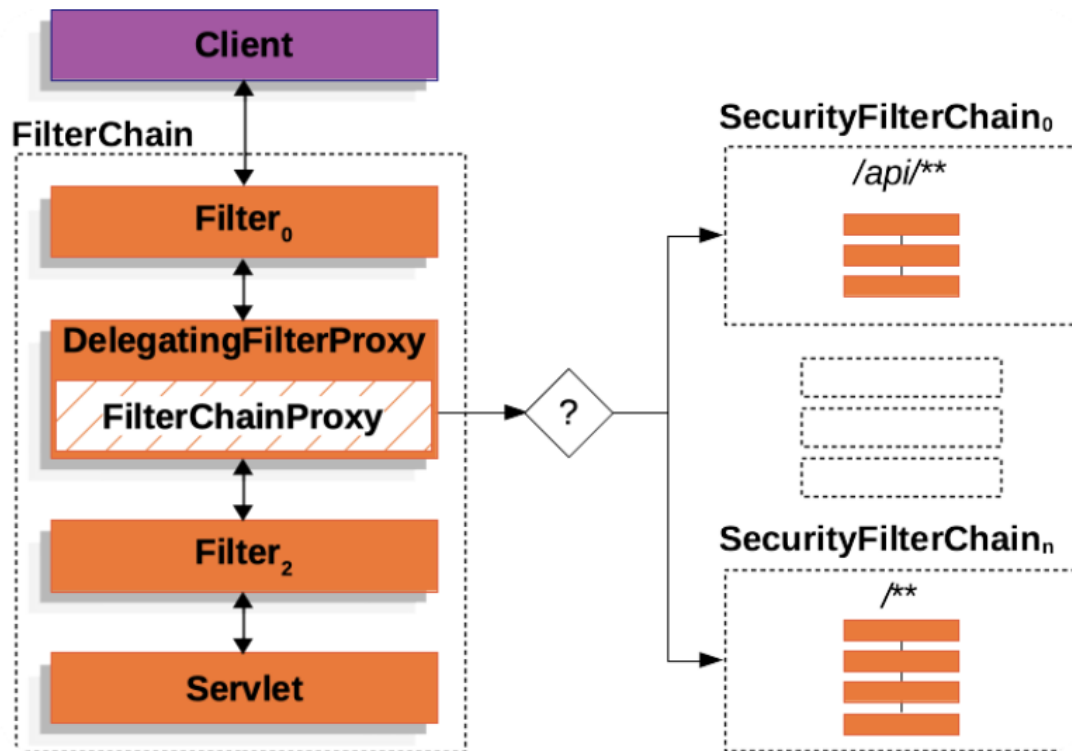


Figure 1b. Delegating the request to the corresponding SecurityFilterChain

If you inspect the FilterChainProxy class, you'd notice how it invokes the security filter chains and in turn trigger the stack of security filters. When a request HttpServletRequest enters SecurityFilterChain, the matches method is used to determine whether the conditions are met to enter the filter chain.

```
// org.springframework.security.web.FilterChainProxy class
public class FilterChainProxy extends GenericFilterBean {
    ...
    private List<SecurityFilterChain> filterChains;
    public FilterChainProxy(List<SecurityFilterChain> filterChains) { this.filterChains = filterChains; }

    private List<Filter> getFilters(HttpServletRequest request) {
        ...
        for (SecurityFilterChain chain : this.filterChains) {
            if (chain.matches(request)) {
                return chain.getFilters();
            }
        }
        return null;
    }
}
```

FilterChainProxy is available by using the @EnableWebSecurity annotation in the (security) configuration

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {
```

Security Filter Chain

The filter chain represents a collection of filters *with a defined order* in which they act. The filters form a chain of responsibilities. They work together to perform various security-related tasks.

A filter receives a request, executes its logic, and eventually delegates the request to the next filter in the chain (figure 1c.).

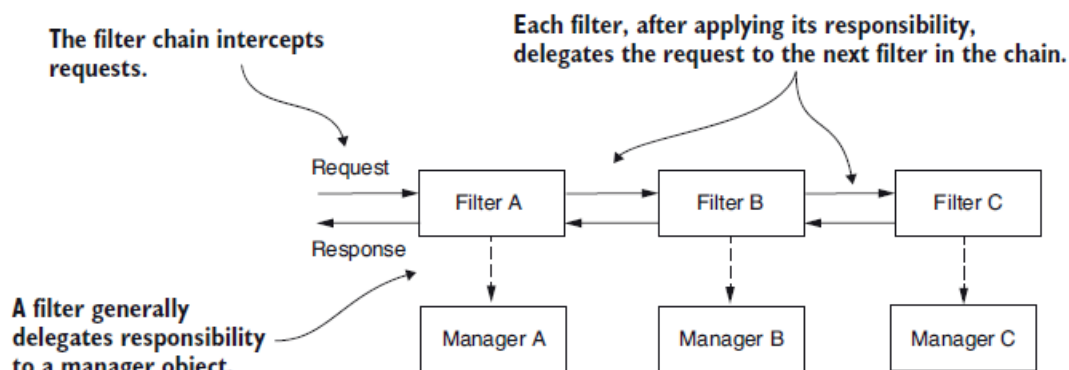


Figure 1c.

In practice, applications come with various requirements where default configurations no longer work. You should adjust the filter chain. Spring Security provides filter implementations that you can enable or disable ; but you can also define custom filters.

Note that there can be 1-n SecurityFilterChains in the application.

The Security Default configuration / defaultSecurityFilterChain

The auto-configuration `SpringBootWebSecurityConfiguration` class provides a default set of Spring Security configurations for Spring Boot applications.

```
// org.springframework.boot.autoconfigure.security.servlet.SpringBootWebSecurityConfiguration
class SpringBootWebSecurityConfiguration {
    ...
    @Bean
    SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
        http.authorizeHttpRequests((requests) -> requests
            .anyRequest().authenticated()
        );
        http.formLogin(withDefaults());
        http.httpBasic(withDefaults());
        return http.build();
    }
}
```

The configuration here is that all requests must be initiated by an authenticated user, with form login and Http Basic Authentication enabled. By default, Spring Security expects the default username „user“. Each time you run the application, it generates a new password and prints this password in the console.

The above configuration will result in the following Filter ordering:

Filter	Added by
UsernamePasswordAuthenticationFilter	HttpSecurity#formLogin (1)
BasicAuthenticationFilter	HttpSecurity#httpBasic (2)
AuthorizationFilter	HttpSecurity#authorizeHttpRequests (3)

Depending on the authentication method, the Authentication-filter (1) or (2) is invoked to authenticate the request. Then the AuthorizationFilter is invoked to authorize the request.

The request is processed by filters *in a order so that authentication occurs before authorization*.

```
UsernamePasswordAuthenticationFilter --
// org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter
```

is a default filter designed for the default form login.

This filter will extract username and password from a POST request body and send them to *ProviderManager* (Authentication Manager). This manager then send them to *DaoAuthenticationProvider* (default Authentication Provider). And this provider will go to *InMemoryUserDetailsManager* (default UserDetailsService), and check if user exists with given credentials.

```
BasicAuthenticationFilter --
// org.springframework.security.web.authentication.www.BasicAuthenticationFilter
```

is responsible for processing any request that has a header "Authorization: Basic Username-Password-Token" with Username-Password-Token - a Base64-encoded *username:password*.

For example, to authenticate user "admin" with password "admin", the following header would be presented "Authorization: Basic YWRtaW46YWRtaW4=".

```
curl -H "Authorization: Basic YWRtaW46YWRtaW4=" localhost:8080/hello
```

If authentication is successful, the resulting Authentication object will be placed into the SecurityContextHolder. If authentication fails and ignoreFailure is false (the default), an AuthenticationEntryPoint implementation is called.

SecurityFilterChain-related Configuration

To configure security at the web request level, we'll need to declare a SecurityFilterChain bean.

```
@Configuration
public class SecuConfig {
    ...
    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            // here call methods of the HttpSecurity object to configure
        ...
        return http.build();
    }
}
```

The filterChain() method accepts an HttpSecurity object, which acts as a builder that can be used to configure how security is handled at the web level. Once security configuration is set up via the HttpSecurity object, a call to build() will create a SecurityFilterChain.

Adding a custom filter in Security Filter Chain

Custom filter can be implemented to handle specific security requirement not covered by the default filters. Custom filter is created by implementing the interface jakarta.servlet.Filter

```
import jakarta.servlet.Filter;
public class CustomFilter implements Filter {
    @Override
    public void doFilter(HttpServletRequest request, HttpServletResponse response, FilterChain chain)
        throws ServletException, IOException { ... }
```

To make sure that our Security filter gets invoked only once for every request. We create class CustomFilter that extends a special filter OncePerRequestFilter.

```
import org.springframework.web.filter.OncePerRequestFilter;

public class CustomFilter extends OncePerRequestFilter {
    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {

        // filter logic
        ...
        // call the next filter
        filterChain.doFilter(request, response);
    }
}
```

Custom filters can be added to the Spring Security filter chain at specific positions relative to existing filters. Methods like `addFilterBefore()`, `addFilterAfter()`, and `addFilterAt()` are used to specify the position of the custom filter in the chain.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    ...
    http.addFilterBefore(new CustomFilter(), UsernamePasswordAuthenticationFilter.class);
    // .addFilterAfter(new LoggingFilter(), UsernamePasswordAuthenticationFilter.class);
    // In some cases, you might want to replace an existing filter with a custom one.
    // For example, you might want to replace the default BasicAuthenticationFilter
    // .addFilterAt(new CustomAuthenticationFilter(), BasicAuthenticationFilter.class)
}
```

Multiple Filter Chains

We can configure *multiple Filter Chains* for application, e.g. one for Web, the other for Rest-API security

```
@Bean
@Order(1)
public SecurityFilterChain apiFilterChain(HttpSecurity http) throws Exception {
    http
        .securityMatcher("/api/**")
        ...
        .httpBasic(withDefaults());
    return http.build();
}

@Bean
public SecurityFilterChain webFilterChain(HttpSecurity http) throws Exception {
    ...
    http.formLogin(withDefaults());
    return http.build();
}
```

2) Authentication

Authentication is the process of verifying who user is.

In a Spring Boot application, this involves checking user credentials (username, password) or token(*) against the stored data, and if they match, the user is granted access to the system. A user has to be first authenticated, for authorization to take place.

(*) In token-based authentication (for example using JSON Web Token), the server doesn't need to store the token. Instead, it accepts the token and *verifies its validity* before granting the user access.

Authentication Architecture in Spring Security

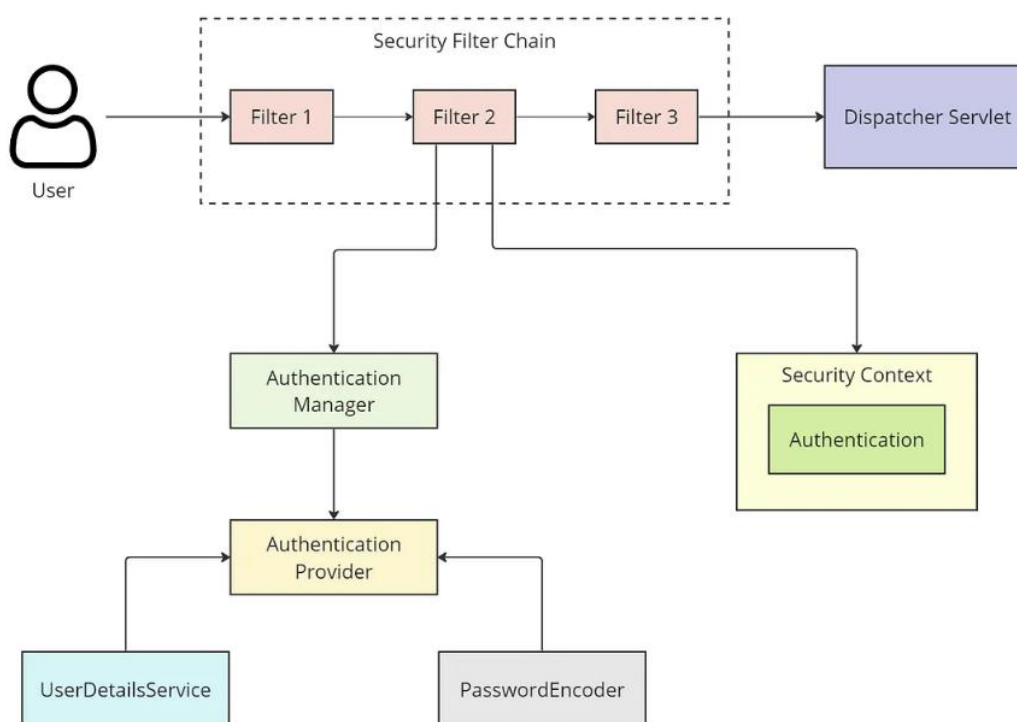


Figure 2a. Authentication Architecture in Spring Security

SecurityContext

The SecurityContext is responsible for storing and retrieving the Authentication object that contains details of the currently authenticated user.

An Authentication filter sets the Authentication object in the SecurityContext

```

Authentication authenticatedUser = ... ;
SecurityContext context = SecurityContextHolder.getContext();
context.setAuthentication(authenticatedUser);

```

The Authorization filter then uses this object to determine access to a specific resource.

```

Authentication authentication = context.getAuthentication();

```

Authentication interface

```

public interface Authentication extends Principal, Serializable {
    Collection<? extends GrantedAuthority> getAuthorities();
    Object getCredentials();
    Object getPrincipal();
    Object getDetails();
    boolean isAuthenticated();
    void setAuthenticated(boolean isAuthenticated) throws IllegalArgumentException;
}

```

UsernamePasswordAuthenticationToken

is a subclass of AbstractAuthenticationToken that implements Authentication, contains information of user.

```
public class UsernamePasswordAuthenticationToken extends AbstractAuthenticationToken {
    private final Object principal; // the user's ID
    private Object credentials;     // the user's PW

    // Create object before authentication is completed
    public UsernamePasswordAuthenticationToken(Object principal, Object credentials) { ... }

    // Create object after authentication is completed
    public UsernamePasswordAuthenticationToken(Object principal, Object credentials,
                                                Collection<? extends GrantedAuthority> authorities) { ... }
}
```

An object created by the first constructor is used as input for authentication.

```
Authentication authen = new UsernamePasswordAuthenticationToken(username, password);
// authen.isAuthenticated() returns false.
```

The result of the authentication is an object created by the second constructor. The user data and authorities are stored in the object.

```
Authentication populatedAuthen = authenticationManager.authenticate(authen);
// populatedAuthen.isAuthenticated() returns true.
```

Authentication Manager

The type AuthenticationManager responsible for authenticating a user.

```
public interface AuthenticationManager {
    Authentication authenticate(Authentication authentication) throws AuthenticationException;
}
```

The authenticate(...) method is passed an Authentication object that contains user credentials (username, password), an Authentication object with user details is returned if successful.

The *ProviderManager* is the default implementation. It manages multiple AuthenticationProviders to handle different authentication mechanisms.

At authentication attempt, ProviderManager iterates through a list of registered AuthenticationProviders, delegates the authentication process to each one in order until one successfully authenticates the user or all providers have been tried.

```
public class ProviderManager implements AuthenticationManager {
    ...
    public List<AuthenticationProvider> getProviders() { return providers; }

    public Authentication authenticate(Authentication authentication) {
        Authentication result = null;

        for (AuthenticationProvider provider : getProviders()) {
            result = provider.authenticate(authentication);
            if (result != null) { ...; break; }
        }
        ...
        return result;
    }
}
```

If any of the providers successfully authenticates the request, the manager returns the Authentication object to the caller (filter). If none of the providers can authenticate the request, the authentication manager will throw an *AuthenticationException*.

Authentication Provider

The authentication provider is the component where *the logic for authentication* resides.

```
public interface AuthenticationProvider {
    Authentication authenticate(Authentication authentication) throws AuthenticationException;
    boolean supports(Class<?> authentication);
}
```

The method `authenticate()` accepts an Authentication object containing user credentials and validates it based on the defined logic.

If the user is successfully authenticated, an another Authentication object with information and authorities of the user is returned. If authentication fails, an *AuthenticationException* is thrown.

The `supports` method of the authentication provider checks whether a particular type of Authentication object can be validated by the authentication provider. This is useful in applications with multiple authentication mechanisms, such as user credentials, access tokens, or OTPs. For each authentication method, individual authentication providers can be defined, ensuring that only requests of the corresponding type are validated by the appropriate provider.

DaoAuthenticationProvider

The AuthenticationProvider interface has around 20 different implementations, each with its own specific functionality. One such implementation is the DaoAuthenticationProvider.

DaoAuthenticationProvider checks whether the provided username exists via *UserDetailsService* and retrieves the user's details, such as the encoded password and any associated roles or permissions. It then uses *PasswordEncoder* to verify that the provided password matches the stored password. If authentication is successful, it returns a fully populated Authentication object with the user's details and granted authorities.

UserDetailsService

This is responsible for retrieving user information from the application's database or other (usually) persistent storage.

```
public interface UserDetailsService {
    UserDetails loadUserByUsername(String username) throws UsernameNotFoundException;
}
```

UserDetailsService Implementation

Solution #1 : Custom Implementation of method : `UserDetails loadUserByUsername(String username)`

Solution #2 : Using (build-in) Classes `JdbcUserDetailsManager`, `InMemoryUserDetailsManager`

PasswordEncoder

This helps to match the provided username and password with the stored ones.

The PasswordEncoder (like `BCryptPasswordEncoder`) contains two methods :

Method `encode()` is used when a user signs up to create a secure version of their password.

Method `matches()` is used during login to check if the password provided by the user matches the encoded password stored in the database.

Custom authentication logic can be implemented by creating a class that implements the `AuthenticationProvider` interface.

```
class CustomAuthenProvider implements AuthenticationProvider {
    @Override
    public Authentication authenticate(Authentication authentication) throws AuthenticationException {
```

The way to register our own `CustomAuthenticationProvider` in the `ProviderManager` described above can be done in `SecurityConfig`.

```
@Configuration
public class AppConfiguration {
    ...

    @Bean
    AuthenticationProvider authenticationProvider() {
        CustomAuthenProvider authProvider = new CustomAuthenProvider();
        return authProvider;
    }

    @Bean
    public AuthenticationManager authenticationManager() {
        return new ProviderManager(List.of(authenticationProvider()));
    }
}
```

Authentication Filter

Authentication filter is the place where the *authentication process* should be implemented.

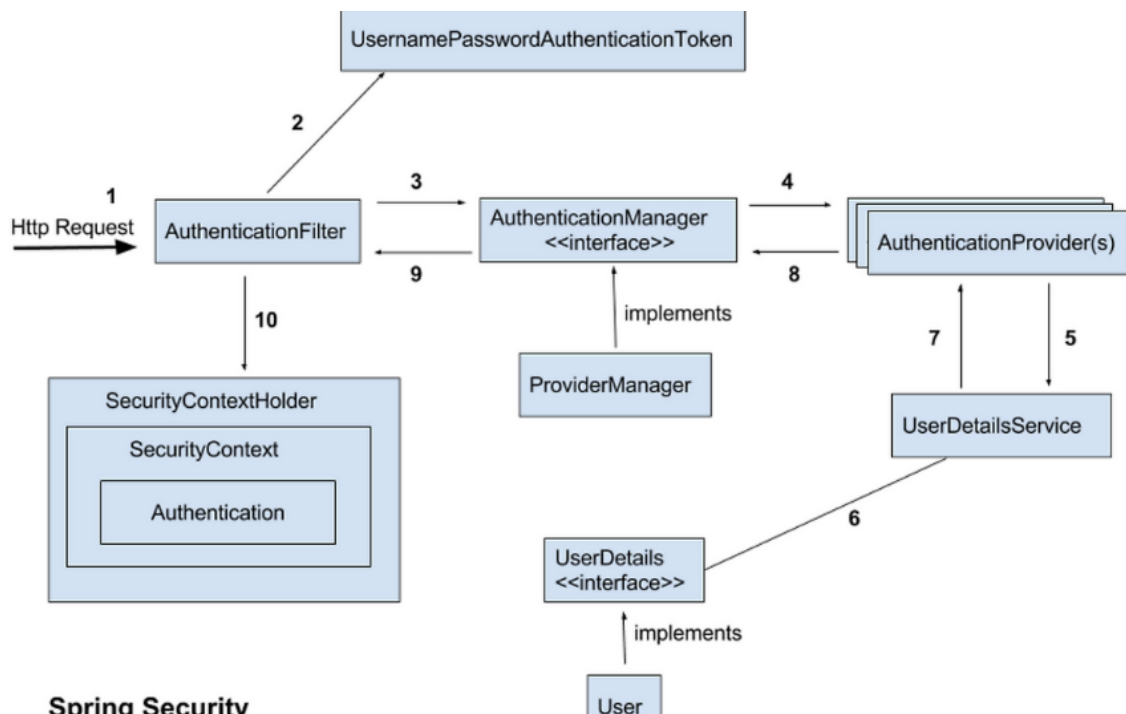


Figure 2b. Authentication process

The filter intercepts the request and *checks whether it is meant to process that particular type of request*.

It then creates an `Authentication` object with the username and password extracted from the request and passes object to the authentication manager.

The authentication manager iterates over the list of authentication providers.

Each authentication provider uses the UserDetailsService to retrieve user information and the password Encoder to validate the password; authenticating the request based on the provided logic.

If the Authentication Manager throws an AuthenticationException, the request is filtered out.

On successful authentication, a fully authenticated Authentication object is returned.

The filter then makes the Authentication object available throughout the lifecycle of the request by feeding it into the SecurityContext.

At any point during the processing of the request, we can retrieve the currently logged-in user's details from this Authentication object by using: SecurityContextHolder.getContext().getAuthentication();

```
public class MyAuthenticationFilter extends OncePerRequestFilter {
    ...
    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain chain) throws IOException, ServletException {
        ...
        Authentication authRequest = this.authenticationConverter.convert(request);

        if (authRequest == null) { chain.doFilter(request, response); return; }

        Authentication existingAuthen = SecurityContextHolder.getContext().getAuthentication();
        // Do not attempt to authenticate if client is already authenticated
        if (existingAuthen != null && existingAuthen.isAuthenticated()) {
            chain.doFilter(request, response); return; }

        try {

            // Perform the authentication
            Authentication authResult = authenticationManager.authenticate(authRequest);

            // and set it in the security context
            SecurityContextHolder.getContext().setAuthentication(authResult);

            // Pass the control to the next filter
            chain.doFilter(request, response);

        } catch (Exception e) {
            handlerExceptionResolver.resolveException(request, response, null, exception);
        }
    }
}
```

3) Authorization

Authorization: The process of determining whether an authenticated user can access a requested resource.

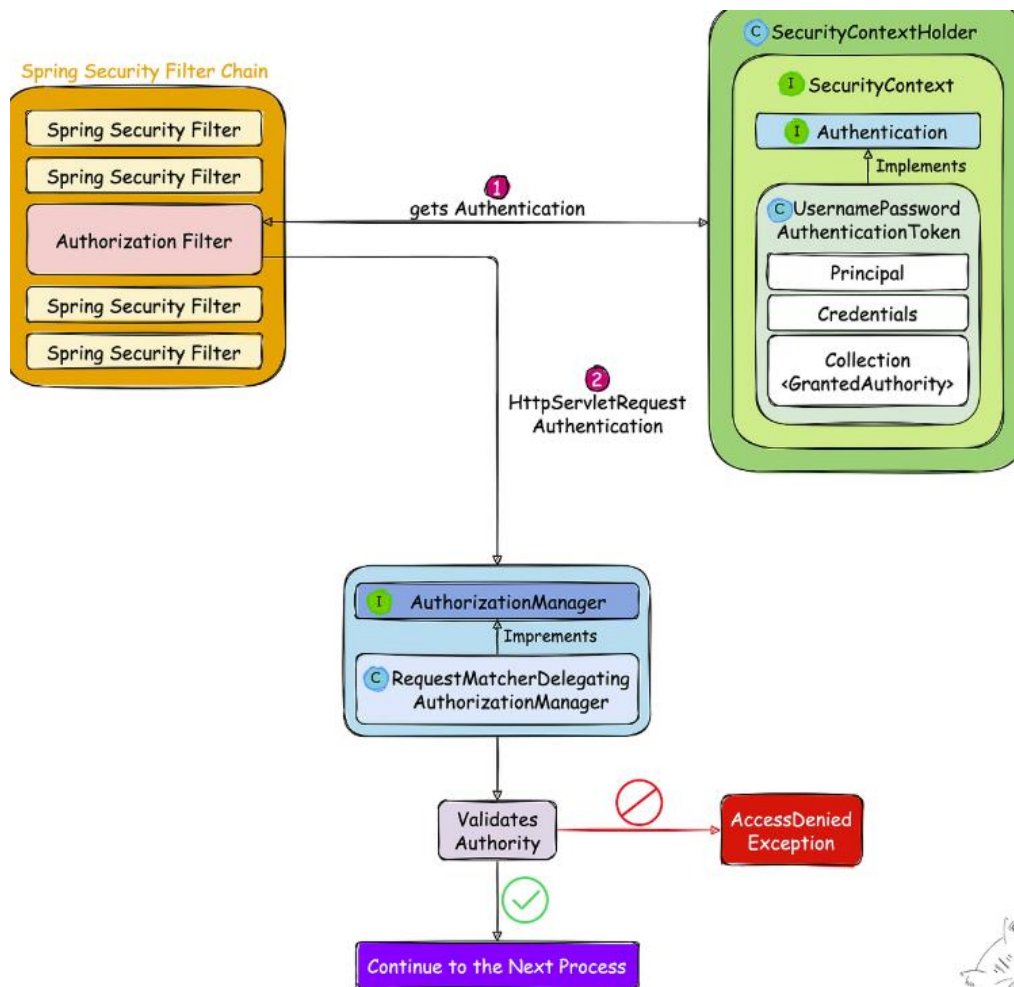


Figure 3a. The Authorization Architecture

AuthorizationFilter

The last filter in the Security filter chain has the function to restrict user access through URL. The decision to allow or deny a request is based on user information (an authentication object stored in the SecurityContext) and endpoint *authorization rules* (declared in the configuration).

```

@Bean
SecurityFilterChain web(HttpSecurity http) throws Exception {
    http.authorizeHttpRequests(authorize -> authorize
        .requestMatchers("/login").permitAll()
        .requestMatchers("/admin/**").hasRole("ADMIN")
        .anyRequest().authenticated()
    );
    return http.build();
}
  
```

If authorization is denied, an **AuthorizationDeniedEvent** is published, and an **AccessDeniedException** is thrown. In this case the **ExceptionTranslationFilter** handles the **AccessDeniedException**.

If access is granted, an **AuthorizationGrantedEvent** is published and **AuthorizationFilter** continues with the **FilterChain** which allows the application to process normally.

The `AuthorizationFilter` doesn't contain much authorization-related logic itself. It merely delegates the authorization request to an `AuthorizationManager`.

```
public class AuthorizationFilter extends OncePerRequestFilter {
    private final AuthorizationManager<HttpServletRequest> authorizationManager;
    ...
    protected void doFilterInternal(HttpServletRequest request, ...) {
        AuthorizationDecision decision =
            this.authorizationManager.check(this::getAuthentication, request);
    }
}
```

AuthorizationManager

In the context of Spring Web MVC, this is per default a *RequestMatcherDelegatingAuthorizationManager*. It receives a `Supplier<Authentication>` and the `HttpServletRequest` that needs to be authorized. It also contains a `List<RequestMatcherEntry>`, which is used to determine if it is responsible for the authorization of the given request. If one of the `RequestMatcherEntry`s matches, its corresponding `AuthorizationManager` is used to do the actual authorization of the request.

The authorization flow

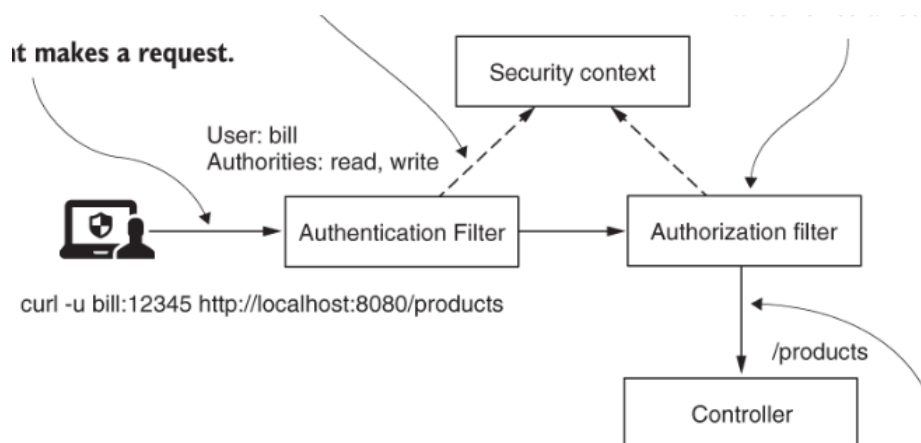


Figure 3b. The Authorization flow

After successful authentication, the authentication filter stores the user details in the Security Context and forwards the request to the authorization filter. The *Authorization filter* gets the user details from the Security Context and decides whether the call is permitted.

4) JWT-token-based Authentication Implement

JWT (JSON Web Token)

There is a standard that is supported by libraries called JSON Web Token (www.rfc-editor.org/rfc/rfc7519).

The header and payload data are signed using either HMAC-SHA-256 (HS256) or RSA-SHA-256 (RS256).

At the end, they are Base64 encoded—and this step generates the JWT.

An example of a JWT

```
{"alg":"HS256","typ":"JWT"}           // header (JSON document), here HS256 Signing Algorithm
{"sub":"1234567890","name":"John Doe", ...} // payload/data (JSON document)
pDx  Lx"UP                             // Signature bytes (*) for header and payload
```

(*) The signature ensures that the client sending the JWT doesn't simply change the token. The client gets a token that was built by someone else and can forward it. Any modification would always be noticed.

Base64 encoded it looks like this:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM4NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c
```

A JWT is Base64 encoded, and there are three parts separated by dots.

Token-based Authentication

With token-based authentication, a token is generated and sent with every request, much like a cookie value. The server accepts the token and verifies its validity before granting access to the user.

JWT-Token-Based Authentication Implement

User Authentication service –

The user data is stored in Database.

The method asUser() is used to convert AppUser object in UserDetails object (4).

```
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
..

@Entity
public class AppUser {
    @Id
    @GeneratedValue(strategy = GenerationType.AUTO)
    private Long id;
    private String username;
    private String password;
    private String role;

    public AppUser(String username, String password, String role) { ... }
    ...
    public UserDetails asUser() { // (4)
        return User.withUsername(getUsername())
            .password(getPassword())
            .roles(getRole())
            .build();
    }
}
```

We get the AppUser by its name as follows

```
public interface AppUserRepository extends CrudRepository<AppUser, Long> {
    Optional<AppUser> findByUsername(String username);
}
```

The (default) DaoAuthenticationProvider requires the UserDetailsService bean and PasswordEncoder bean for his work (1)(2). For authentication we need AuthenticationManager bean (3)

Together we need to configure the following

```
@Configuration
public class AppConfig {
    ...
    private final AppUserRepository userRepository;

    @Bean
    UserDetailsService userService() { // (1)
        return username -> userRepository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found"))
            .asUser();
    }
    @Bean
    public PasswordEncoder passwordEncoder() { // (2)
        return new BCryptPasswordEncoder();
    }
    @Bean
    public AuthenticationManager authenticationManager(AuthenticationConfiguration authConfig)
        throws Exception { // (3)
        return authConfig.getAuthenticationManager();
    }
}
```

JWT-Token service –

For token generation and token validity we need JwtService class with 2 operations

```
@Service
public class JwtService {

    @Value("${security.jwt.secret-key}")
    private String secretKey;

    @Value("${security.jwt.expiration-time}")
    private long jwtExpiration;

    // Generate signed JWT token
    public String getToken(String username) { ... }

    // verify thea token, and get username
    public String getAuthUser(String token) { ... }
```

JWT token generation (Login) ---

The user must log in with a username and password, and if the credentials are correct, the response is the JWT.

```

@RestController
public class LoginController {
    ...
    @PostMapping("/login")
    public ResponseEntity<?> getToken(@RequestBody AccountCredentials credentials) {
        ...
        // User authentication
        Authentication auth = authenticationManager.authenticate(creds);
        // Generate token
        String jwt = jwtService.getToken(auth.getName());
        ...
    }
}

```

JWT-Filter / Authentication process ---

The client sends request with the header "Authorization: Bearer JWT" to the server.

A request could look like this: GET http://localhost:8080/ Authorization: Bearer JWT

The filter checks whether it is responsible for handling this request. If not, the request is forwarded to the next filter (1)

The JWT service then verifies the jwt-token. If the token is valid, username is extracted. (2)

Next, we reset the authenticated user (Authentication object) in SecurityContext if necessary (3)

Finally, we call filterChain.doFilter(request, response) so that the next filter gets called in the FilterChain (4)

```

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final JwtService jwtService;
    private final UserDetailsService userDetailsService;
    private final HandlerExceptionResolver handlerExceptionResolver;
    ...

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
        FilterChain filterChain) throws ServletException, IOException {

        final String authHeader = request.getHeader("Authorization");

        if (authHeader == null || !authHeader.startsWith("Bearer ")) { // (1)
            filterChain.doFilter(request, response); return;
        }

        try {

            final String jwt = authHeader.replace(PREFIX, "");

            // Verify token and get username
            String username = jwtService.getAuthUser(jwt); // (2)

            // reset the authenticated user if necessary
            Authentication authen = SecurityContextHolder.getContext().getAuthentication();
        }
    }
}

```

```

if (authen == null || !authen.getName().equals(username)) { // (3)
    UserDetails userDetails = userDetailsService.loadUserByUsername(username);
    authen = new UsernamePasswordAuthenticationToken(username, null, userDetails.getAuthorities());
    SecurityContextHolder.getContext().setAuthentication(authen);
}

// go to next filter
filterChain.doFilter(request, response); // (4)

} catch (Exception ex){ // (5)
    handlerExceptionResolver.resolveException(request, response, null, ex);
}

}
}

```

SecurityConfiguration ---

```

@Configuration
@EnableWebSecurity
public class SecuConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            // .authenticationProvider(authenticationProvider)
            .addFilterBefore(JwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class)
    }
}

```

Secure Exception Handling ---

```

@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
        ...
        try { // Throw exceptions
            ...
        } catch (Exception exception) { // pass to ExceptionResolver
            handlerExceptionResolver.resolveException(request, response, null, exception);
        }

    // Handling
    @RestControllerAdvice
    public class GlobalExceptionHandler {

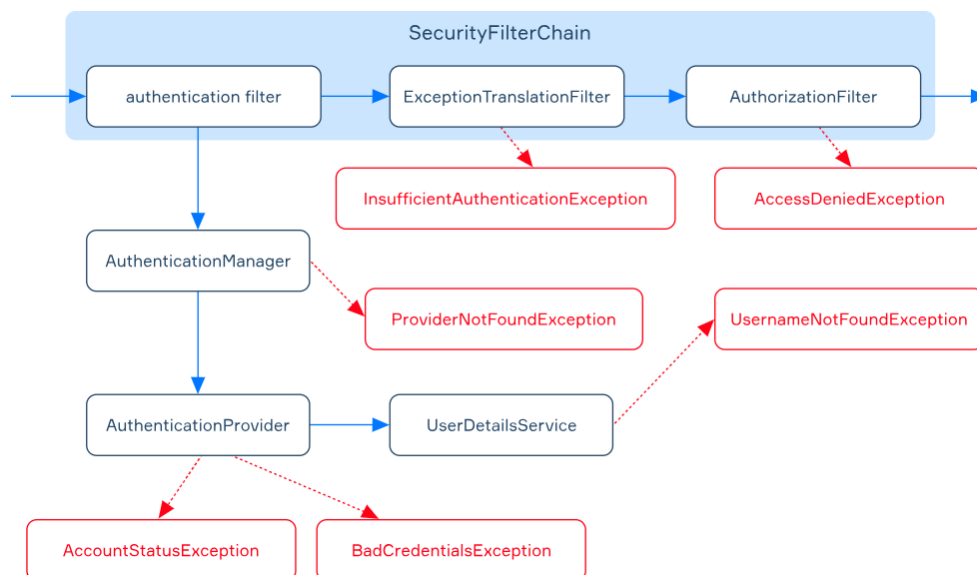
        @ExceptionHandler(Exception.class)
        public ProblemDetail handleSecurityException(Exception exception) {
    }
}

```


5) Handling Security Exceptions

Spring Security divides exceptions into two categories: authentication-related and authorization-related. The base exception for authentication-related issues is *AuthenticationException*, an abstract class that extends *RuntimeException*. There are numerous concrete implementations that extend *AuthenticationException*. For authorization-related issues, the base class is *AccessDeniedException*, a concrete class that also extends *RuntimeException*.

Let's explore some of these exceptions that basic HTTP authentication can throw.



Exception Handling in Spring Filters

Effective Strategy : Manual Exception Handling within the Filter

Directly catch exceptions within the `doFilter` method of your filter.

Log the exception details for debugging purposes.

Set appropriate HTTP status codes on the response object (e.g., 400 Bad Request, 500 Internal Server Error).

Optionally, write a custom error response body in the desired format (JSON, XML, etc.).

Code Example:

```

@Component
public class CustomFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {
        try {
            // Filter logic...
            filterChain.doFilter(request, response);
        } catch (Exception e) {
            logger.error("Error during filtering:", e);
            response.setStatus(HttpServletResponse.SC_BAD_REQUEST); // Set appropriate status code
            response.getWriter().write("An error occurred while processing your request.");
        }
    }
}
  
```

ExceptionTranslationFilter

handles any *AccessDeniedException* and *AuthenticationException* thrown within the filter chain. This filter is necessary because it provides the bridge between Java exceptions and HTTP responses. It is solely concerned with maintaining the user interface.

The pseudocode for *ExceptionTranslationFilter* looks something like this:

```
try {
    filterChain.doFilter(request, response);
} catch (AccessDeniedException | AuthenticationException ex) {
    if (!authenticated || ex instanceof AuthenticationException) {
        startAuthentication();
    } else {
        accessDenied();
    }
}
```

If an *AuthenticationException* is detected, the filter will launch the *AuthenticationEntryPoint*.

If an *AccessDeniedException* is detected, the filter will determine whether or not the user is an anonymous user. If they are an anonymous user, the *AuthenticationEntryPoint* will be launched. If they are not an anonymous user, the filter will delegate to the *AccessDeniedHandler*.

AuthenticationEntryPoint

By default, the *BasicAuthenticationEntryPoint* returns a full page for a 401 Unauthorized response back to the client. To customize the default authentication error page used by basic auth, we can implement the *AuthenticationEntryPoint* interface.

```
@Component
public class CustomAuthenticationEntryPoint implements AuthenticationEntryPoint {
    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response,
        AuthenticationException authException) throws IOException, ServletException {

        response.setStatus(HttpServletResponse.SC_UNAUTHORIZED);
        response.getWriter().write("Bad Credentials.");
    }
}
```

AccessDeniedHandler

To customize the access refusal used by basic auth, we can implement the *AccessDeniedHandler* interface.

```
@Component
public class CustomAccessDeniedHandler implements AccessDeniedHandler {
    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
        AccessDeniedException accessDeniedException) throws IOException, ServletException {
        response.setStatus(HttpServletResponse.SC_FORBIDDEN);
        response.getWriter().write("Access Denied. You do not have privileges to access this resource.");
    }
}
```

We register the above merchants as following

```
@Configuration
public class WebSecurityConfig {
    @Autowired
    @Qualifier("customAuthenticationEntryPoint")
    private AuthenticationEntryPoint authEntryPoint;

    @Autowired
    @Qualifier("customAccessDeniedHandler")
    private AccessDeniedHandler accessDeniedHandler;
    ...

    @Bean
    SecurityFilterChain apiFilterChain(HttpSecurity http) throws Exception {
        http
            // in basic authentication ----
            .httpBasic(basic -> basic.authenticationEntryPoint(authEntryPoint))
            .exceptionHandling(customizer -> customizer.accessDeniedHandler(accessDeniedHandler))
            ;
        // or for custom filter e.g. jwtAuthenticationFilter ----
        .addFilterAfter(jwtAuthenticationFilter, ExceptionTranslationFilter.class)

        .exceptionHandling(exception -> exception
            .authenticationEntryPoint(authEntryPoint)
            .accessDeniedHandler(accessDeniedHandler)
        )
        ;
    }
}
```

Handling security exceptions with @ExceptionHandler and @ControllerAdvice

This approach allows us to use exactly the same exception handling techniques but in a cleaner and much better way in the controller advice with methods annotated with @ExceptionHandler.

Spring security core exceptions such as AuthenticationException and AccessDeniedException are runtime exceptions. Since these exceptions are thrown by the authentication filters *before* invoking the controller methods, @ControllerAdvice won't be able to catch these exceptions. To handle these exceptions at a global level via @ExceptionHandler and @ControllerAdvice, we need delegate the exception to *HandlerExceptionResolver*.

The adjustment at filter

```
@Component
public class JwtAuthenticationFilter extends OncePerRequestFilter {

    private final HandlerExceptionResolver handlerExceptionResolver;

    @Override
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                    FilterChain filterChain) throws ServletException, IOException {
        try {
            ...
            filterChain.doFilter(request, response);
        } catch (Exception exception) {
            handlerExceptionResolver.resolveException(request, response, null, exception);
        }
    }
}
```

Or a new custom implementation of `AccessDeniedHandler`:

```
@Component
public class DelegatedAccessDeniedHandler implements AccessDeniedHandler {
    @Autowired
    @Qualifier("handlerExceptionResolver")
    private HandlerExceptionResolver resolver;

    @Override
    public void handle(HttpServletRequest request, HttpServletResponse response,
        AccessDeniedException accessDeniedException) throws IOException, ServletException {
        resolver.resolveException(request, response, null, accessDeniedException);
    }
}
```

A new custom implementation of `AuthenticationEntryPoint`:

```
@Component
public class DelegatedAuthenticationEntryPoint implements AuthenticationEntryPoint {
    ...
    @Override
    public void commence(HttpServletRequest request, HttpServletResponse response,
        AuthenticationException authException) throws IOException, ServletException {
        resolver.resolveException(request, response, null, authException);
    }
}
```

Now we can handle the exceptions in the class annotated with `@RestControllerAdvice`.

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(AuthenticationException.class)
    public ErrorDetails handleAuthenticationException(Exception e) {
        ...
        errorDetails.setMessage(e.getMessage());
        return errorDetails;
    }

    @ExceptionHandler(AccessDeniedException.class)
    public ErrorDetails forbidden(Exception e) {
        ...
        errorDetails.setMessage(e.getMessage());
        return errorDetails;
    }
}
```

6) OAuth2 INTRO

Cryptography

signature

Data --HashF--> Hash --RSA-privateKey-encoded (sign)--> Signature

The signature is used *to verify the authenticity and integrity of the token*

Token

Data + Signature = token

Token validation

Signature --RSA-publicKey-decoded--> Hash#1

Data --HashF--> Hash#2

Hash#1 == Hash#2 ? if true OK else NOT ok

certificate

(Data + publicKey) ... --> Signature

(Data + publicKey) + Signature = Certificate

PKI Infrastruktur (for TLS/SSL/HTTPS communication)

CA-Certificate <=> publicKey of CA

Server-Certificate <=> publicKey of Server signed by CA

CA issues Server-Certificate

TLS/SSL/HTTPS communication

client --- server

Client validates Server-Certificate

Client create a sessionKey (symmetri)

Client -- Server use sessionKey for secu communication

OAuth2 in Spring Security

OAuth System

Client: This is the application or service that wants to access the resource.

Authorization Server: Responsible for generating and providing *access token* (JWT token) to the client and verifying whether an access token is valid.

Resource server: The server that stores or hosts the resource.

Client <---> Authorization Server : get access token

Client <---> Resource server : using access token, token validation

JWT token (JSON Web Tokens)

A JWT consists of three parts: a header, a payload, and a signature. These parts are encoded in base64 and are separated by period (.) signs:

The header contains the metadata needed to manage the token.

The payload carries information such as the custom claims and expiration time, issuer.

The signature is created by taking the header and payload as a hash value, and signing it using the signing algorithm specified in the header.

The signature is used *to verify the authenticity and integrity of the token* so that we can ensure the token was issued by the authorization server and was not modified by anyone else.

Fetching access token

grant type : The process of /fetching this access token/ is called authorization flow or /grant type/.

Grant Types --

Authorization Code --

The *authorization code* grant type/flow is used when dealing with some kind of *user interface* where it's possible to authenticate using a web browser.

```
user authen : client-id + user credentials    =>  code
client authen: client-id + client-secret + code => access token
```

Client Credentials --

But in the case of REST APIs, this is not feasible because the client wants to authenticate silently without filling out any login form.

For this reason, you can use the *client credentials* grant type/flow.

```
client authen: client-id + client-secret => access token
```

Token validation

In Client --

If your provider (authorization server) fulfills the OpenID Connect protocol correctly, you only need to configure the issuer URI.

The app then uses the issuer URI to find all details it needs, such as the authorization, token, and the key set URI.

```
config: issuer-uri (== Authen Server URI ) --> public key ----> validation token
// example
spring.security.oauth2.client.provider.springblog.issuer-uri=http://127.0.0.1:7070
```

In Resource server --

The Resource Server contacts the Authorization Server to verify whether the access token is valid.

For this reason, you need to define the URL of the Authorization Server inside the Resource Server.

This is the ***JWKS (JSON web key set) URL of the Authorization Server***, which contains the information about the keypair used to sign the access token.

Spring Security will read this information during the application startup to set up the required JWKS to verify the incoming access token.

The JWK Set URL is exposed as part of the discovery document <http://localhost:9000/.well-known/oauth-authorization-server>.

```
config: jwk-set-uri
// example
spring.security.oauth2.resourceserver.jwt.jwk-set-uri=http://localhost:9000/oauth2/jwks
```

7) Spring Security Configuration without Boot

To configure Spring Security for Spring Web Mvc the developer must take care of three things.

- declare the security filter for the application
- define the Spring Security context
- configure authentication and authorization

If we use Spring MVC, our `SecurityWebApplicationInitializer` could look something like the following:

```
public class SecurityWebApplicationInitializer extends AbstractSecurityWebApplicationInitializer {}
```

This only registers the *DelegatingFilterProxy* (a servlet-filter) with Servlet Container for every URL in your application.

After that, we need to ensure that `WebSecurityConfig` was loaded in our existing `ApplicationInitializer`.

For example, if we use Spring MVC it is added in the `getServletConfigClasses()`:

```
public class MvcWebApplicationInitializer extends AbstractAnnotationConfigDispatcherServletInitializer
{
    ...
    @Override
    protected Class<?>[] getServletConfigClasses() {
        return new Class[] { WebSecurityConfig.class, WebMvcConfig.class };
    }
}
```

And configuring the Web Security

```
@Configuration
@EnableWebSecurity
public class WebSecurityConfig {

    @Bean
    public PasswordEncoder encoder() {
        ...
    }

    @Bean
    public UserDetailsService userDetailsService(DataSource dataSource) {
        ...
    }

    @Bean
    @Order(1)
    public SecurityFilterChain apiFilterChain(HttpSecurity http) throws Exception {
        ...
        return http.build();
    }
}
```

LINKS ----

// good architect //

<https://velog.io/@gwon477/Spring-Security-JWT>

// good architect //

https://jschmitz.dev/posts/exploring_spring_securitys_authentication_mechanism_in_web_applications/

https://jschmitz.dev/posts/exploring_spring_securitys_authorization_mechanisms/

// good Authentication //

<https://medium.com/@sallu-salman/understanding-the-key-components-in-spring-security-authentication-245297a40b93>

<https://devsyw.tistory.com/11>

// deeper //

<https://backendstory.com/spring-security-authentication-architecture-explained-in-depth/>

// good Authorizing //

<https://gnidinger.tistory.com/entry/SpringSpring-Security-%EA%B6%8C%ED%95%9C-%EB%B6%80%EC%97%ACAAuthorization-%EA%B5%AC%EC%A1%B0>

<https://szarpcode.com/spring-security-fundamentals/>

// good book Spring Secu in Action //

<https://fizalihan.github.io/technology/spring-security.html>

<https://www.codejava.net/frameworks/spring/handle-exceptions-in-a-filter>

<https://medium.com/@tharinduanupa05/custom-exception-handling-for-spring-security-related-exceptions-dcc93fcb187a>

// good JWT Authentication Implement //

<https://medium.com/@tericcabrel/implement-jwt-authentication-in-a-spring-boot-3-application-5839e4fd8fac>

// Implement //

<https://www.codingshuttle.com/blogs/spring-security-is-really-not-that-hard-internal-working-of-spring-security/>

https://scrutinybykhimaanshu.blogspot.com/2020/02/spring-security-part-3-spring-security.html#google_vignette